

GUÍA DE LABORATORIO 08

160005042 - Carlos González
160005116 - David Garzón
160005105 - Nicolás Cabrera
160005217 - Juan Leguizamón

- **Diagnóstico inicial**

Identifica con claridad problemas de responsabilidad, acoplamiento y organización en el proyecto original.

El problema principal de responsabilidad que tenía el proyecto original se miraba en el Main, ya que este tenía demasiadas responsabilidades mezcladas en el switch, tenía dentro de este 5 responsabilidades, como mostrar el menú, la búsqueda de usuarios, la validación general del parqueadero, las acciones de ingresar, salir y calcular, como también el cálculo de la tarifa. Por el lado del acoplamiento el Main conocía y dependía de las clases Usuario, Vehículo, Ticket, Carro, Moto y CalculadoraTarifa, problema grave por el hecho de que un cambio en una de estas podía romper el Main, el proyecto no manejó una estructura organizada de los archivos, todos estaban en el mismo espacio de la carpeta del proyecto haciéndolo una sopa.

- **Decisiones de refactorización**

Se aplicó una arquitectura basada en capas (Arquitectura Limpia) para dividir las responsabilidades del software:

A. Capa de Dominio (src.entities):

- Se aislaron las clases Vehículo, Carro, Moto, Usuario y Ticket.
- Mejora clave: El Usuario ahora posee una List<Vehículo>. Esto permite que el sistema soporte que un cliente tenga múltiples vehículos registrados, eliminando la restricción 1-a-1 de la versión anterior.

B. Capa de Aplicación (src.usecases):

- Se extrajo la lógica de los procesos y se convirtió en "Casos de Uso" independientes como IngresarVehículoUseCase y SalidaVehículoUseCase.
- Esto permite que la lógica de "cómo funciona el parqueadero" sea universal y no dependa de quién la llame.

C. Capa de Puertos (src.ports):

- Se definió la interfaz UserRepo. Esta actúa como un contrato que describe qué operaciones de datos necesita el sistema, sin especificar cómo se hacen.

D. Capa de Infraestructura (src.infrastructure):

- Aquí se implementó UserRepoImp para manejar la persistencia en memoria y la CalculadoraTarifa.

- Si en el futuro se desea usar una base de datos real, solo se debe crear una nueva implementación en esta capa sin tocar el resto del código.

E. Capa de Adaptadores (src.adapters):

- Se creó MenuConsole para separar la interacción con el usuario (Scanner y menús) del flujo lógico del programa.

• Justificación.

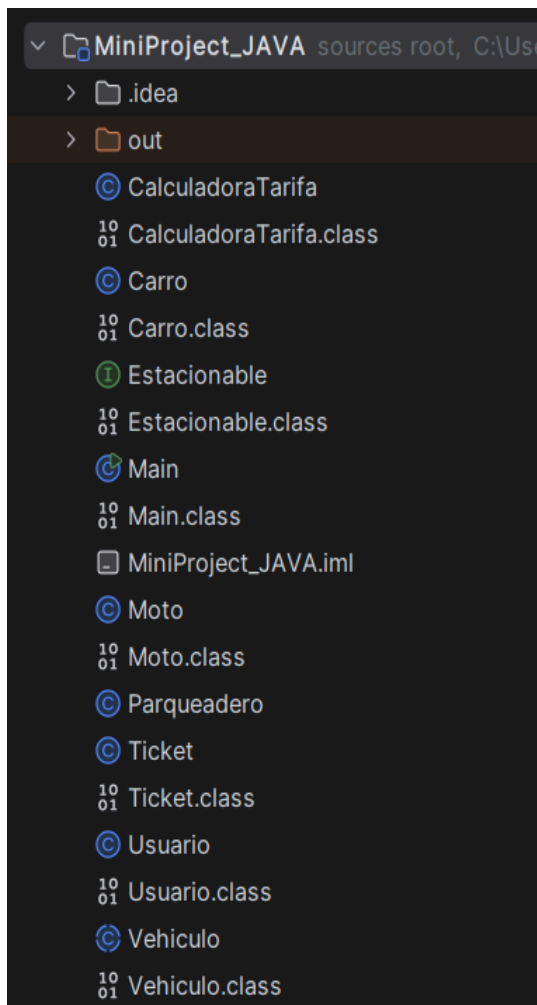
Mantenibilidad: Ahora es posible cambiar la lógica de cobro o los mensajes de la interfaz de forma independiente. Si hay un error en el cálculo, se busca en los UseCases; si hay un error en el menú, se busca en los Adapters.

Inversión de Dependencias: Los componentes de alto nivel (el negocio) ya no dependen de los de bajo nivel (la consola o las listas). Ahora ambos dependen de abstracciones (interfaces).

Código Limpio (Clean Code): El archivo Main.java dejó de ser un archivo de cientos de líneas para convertirse en un iniciador simple que orquesta el arranque del sistema.

Escalabilidad: El sistema está preparado para crecer. Agregar nuevos tipos de vehículos o métodos de pago es ahora una tarea de extensión y no de modificación del código existente.

• Árbol de paquetes INICIO



- Árbol de paquetes FINAL

